



COSCUP 2026



Ruby Taiwan

Building Native Mobile Apps Using Rails

Ship iOS and Android from the app you already have — with Hotwire Native.

● Speaker: **Neba**



Aug 8, 2026



NTUST, Taipei, Taiwan



哈羅👋, 我是 Neba



7+ Years Experience



Ethiopia 🇪🇹 → Taiwan



ASUS & 508.dev Co-op



Shipped & Building with Hotwire Native



Rubyist



OS & Environment: I use omarchy btw



nebiyu@508.dev



github.com/nebiyuelias1



[Starter Template](#)

Turning an Idea into a Product 🚀



🚀 The Rails Advantage: Turns raw ideas into shipping products faster than any other framework.

0 to 1 Velocity 💎

Traction Hits... & Users Demand **Mobile!**



"Is there an App Store link?"

App Store Discovery



"Send me Push Notifications!"

Native Engagement



"I want an icon on my Home Screen!"

User Retention



The Reality: As your web product grows, shipping native iOS & Android apps becomes mandatory for survival.

Mobile Demand



Why Mobile Apps Are Required

88%

Mobile Time Spent in Apps

Users spend 88% of mobile time inside native apps vs. only 12% on mobile web browsers.

3.5x

Higher User Retention

Native app users return 3.5x more frequently compared to mobile website visitors.

3.0x

Higher In-App Conversion

In-app checkout and subscription conversion rate is 3x higher than mobile web forms.

70%+

Expect Push & Home Screen Icon

Over 70% of active mobile users expect instant lock-screen alerts and 1-tap icon access.

1. Progressive Web Apps (PWAs)

⚠ Low Setup / High Friction



No App Store Presence

Users search App Store & Play Store — not browser web links.



iOS Safari Quirks

Flaky push notifications & strict WebKit storage limits.



Web-Only Feel

Lacks native 60fps animations, native sheets & system tabs.

💡 **Verdict:** Easiest to deploy, but weak user reach & retention.

Level 1 Complexity

2. React Native / Flutter

 Medium / API & State Tax



Heavy JSON API Tax

Must write, version, and sync
REST/GraphQL endpoints for every screen.



Double State Management

Database state on server vs client JS state
(Redux/Zustand) on mobile.



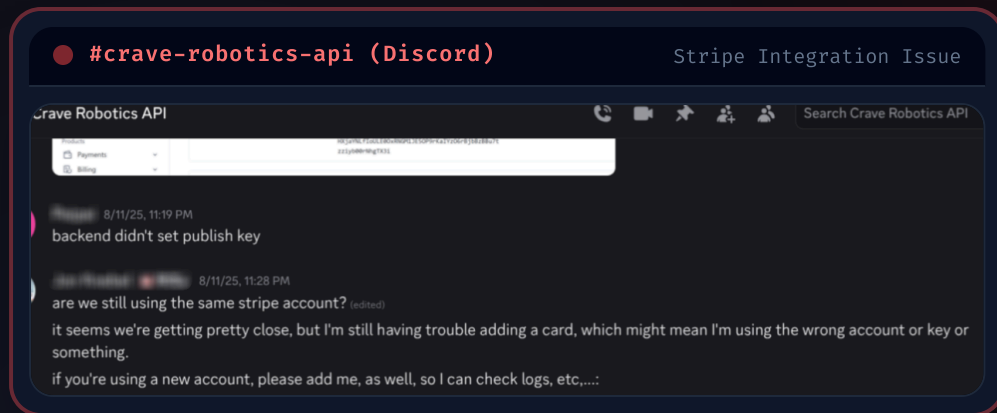
Discards Server HTML

Completely bypasses existing Rails views,
ERB templates & helpers.

 **Verdict:** Cross-platform UI, but doubles backend & state overhead.

Level 2 Complexity

Real Story: The React Native API Tax ⚡



⚠ Actual Discord debug log from my client project with Crave Robotics

⚡ API Key & Environment Desync

Backend changed Stripe publishable keys & accounts, breaking card tokenization silently on the React Native mobile app.

🗣 Communication Overhead

Hours wasted cross-checking API logs between backend devs and mobile React Native devs just to align endpoints and keys.

3. Separate Native Apps (Swift & Kotlin)

● Highest / Tripled Cost



3 Codebases

Rails Monolith + Swift (iOS) + Kotlin (Android).



Tripled Engineering Team

Requires dedicated iOS & Android engineers for simple updates.



Duplicated Logic

Form validations & business workflows must be written 3 times.

💡 **Verdict:** 100% native performance, but unsustainable cost for small teams.

Level 3 Complexity

But Wait... We're a **Small Team!** 🚀

What if we can use the Rails app we already have to also build our mobile app?

Then The "Cool Kids" Speak Up... 🙄



🎬 Mean Girls (2004)

Mobile Dev Community Dogma

Embrace Being Contrarian 🎯

1. Solid Cache & Solid Queue



Database as Infrastructure

"You must run Redis, Memcached, and complex RAM server clusters just for caching and background jobs."

Use your existing SQL database for caching (SolidCache) and background jobs (SolidQueue). Zero extra RAM clusters, zero extra DevOps maintenance.

Embrace Being Contrarian 🎯

2. Schema Introspection



ActiveRecord Runtime Magic

"Manually define, duplicate, and sync every database model property across ORMs, DTOs, and type definitions."

ActiveRecord inspects the database schema automatically at boot time. Zero boilerplate model declarations, zero schema drift, and maximum developer efficiency.

Embrace Being Contrarian 🎯

3. Majestic Monolith



1 Developer Powerhouse

"Split your early app into 50 Kubernetes microservices with distributed tracing and API gateways."

One cohesive Rails monolith contains all your business logic. 1 developer can build, test, and deploy features in minutes instead of waiting on cross-team coordination.

Embrace Being **Contrarian** 🎯

4. HTML Over The Wire (Hotwire)



Web + Mobile Unified

~~"Build separate SPA frontend apps, REST API layers, and rewrite all UI components natively for iOS & Android."~~

Render HTML on the server and stream directly inside native mobile shells. Power your Web app, iOS app, and Android app simultaneously from 1 Rails codebase!

Enter Hotwire Native ⚡



Built by 37signals

Open-source framework built for HEY & Basecamp.

Battle-tested in production 🛡️



Send HTML to Mobile

Server renders HTML views directly inside native mobile shells.

No JSON APIs required ⚡



Logic Stays on Server

All business logic remains in Rails. Mobile apps are thin clients.

Zero code duplication ©



Write Ruby. Render HTML. Deploy to server — instantly live on Web, iOS & Android.

Single Majestic Monolith ⚡

What is Hotwire Native?

Thin Native Shell

Native Swift & Kotlin container managing top navbars, bottom tab bars, and push notifications.

iOS & Android Shell

Renders Web Views

Reuses 100% of your existing Rails ERB views and business logic with zero code duplication.

Single Majestic Monolith

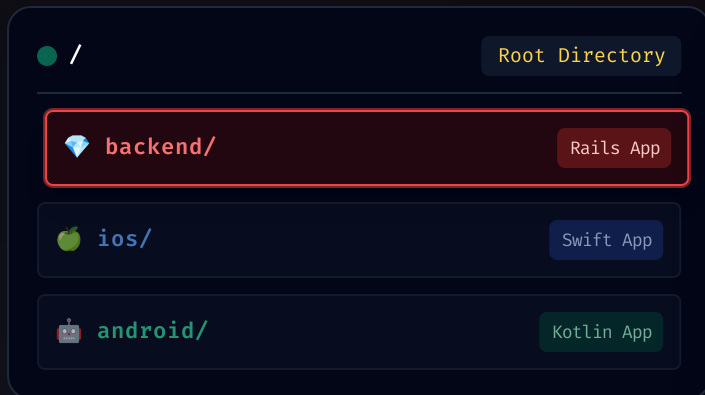
⚡ Native Where It Matters

Seamlessly drop down to 100% native screens for camera, biometrics, or heavy custom components.

Frictionless Native Fallback

Monorepo Setup: Rails + iOS + Android

backend/ - Rails Monolith Structure



backend/

```
├─ app/  
  └─ controllers/  
      └─ configurations_controller.rb ⚡ (Serves Path Config JSON)  
          └─ views/ (Renders HTML for Web & Mobile)  
├─ config/routes.rb  
└─ Gemfile
```

```
get "/configurations/ios_v1", to: "configurations#ios_v1"
```

 Rails Backend: Houses models, controllers & ERB views + serves dynamic navigation rules for mobile.

Adding Hotwire Native to iOS (Swift)

Step 1

Step 2

Step 3

1. Define Base Server URL

Set your Rails app base URL endpoint.

2. Initialize Hotwire Navigator

Create `Navigator(configuration:)` with start location.

3. Connect Window & Route

Set `rootViewController` and trigger `route(baseURL)`.

ios/App/Delegates/SceneDelegate.swift

Step 1 of 3

```
import HotwireNative
import UIKit

let baseURL = URL(string: "https://litloop.club/")!

class SceneDelegate: UIResponder, UIWindowSceneDelegate {
    var window: UIWindow?

    private let navigator = Navigator(configuration: .init(
        name: "main", startLocation: baseURL.appending(path: "/")
    ))

    func scene(_ scene: UIScene, willConnectTo ...) {

        window?.rootViewController = navigator.rootViewController
        navigator.route(baseURL)

    }
}
```

Adding Hotwire Native to Android (Kotlin)

Step 1

Step 2

Step 3

1. Set Base Server URL

Define your Rails app production URL endpoint.

2. Subclass HotwireActivity

Extend `HotwireActivity` in your `MainActivity.kt`.

3. Configure Navigator Location

Set `startLocation` in `NavigatorConfiguration`.

`android/app/src/main/java/et/netale/litloop/MainActivity.kt`

Step 1 of 3

```
package et.netale.litloop
import dev.hotwire.navigation.activities.HotwireActivity
import dev.hotwire.navigation.navigator.NavigatorConfiguration

const val baseUrl = "https://litloop.club"

class MainActivity : HotwireActivity() {

    override fun navigatorConfigurations():
        List<NavigatorConfiguration> = listOf(

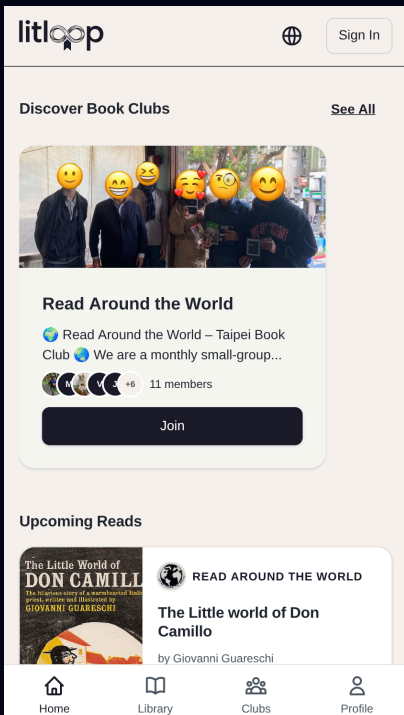
            NavigatorConfiguration(
                name = "main", startLocation = "$baseUrl", navigatorHostId
                = R.id.main
            )

        )
}
```

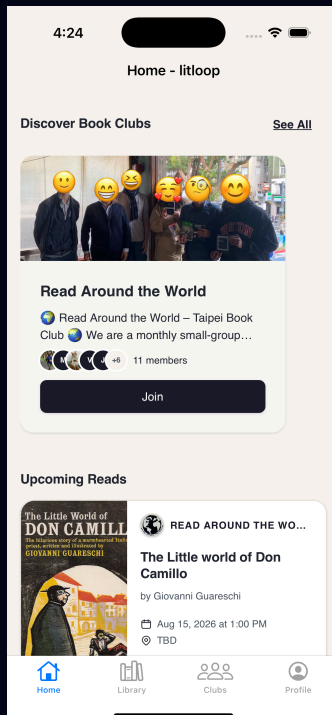
The Result: Native + HTML hybrid

🎯 Highlight Native vs HTML

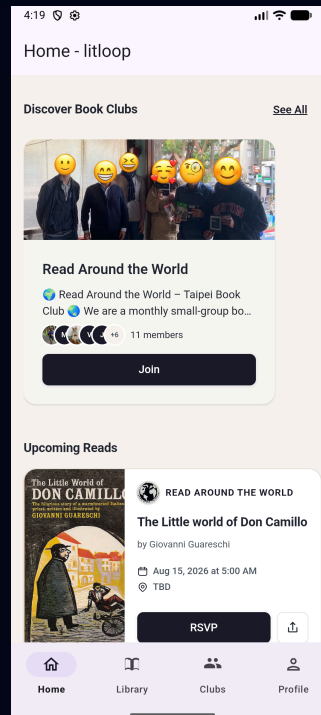
Web Browser



Native iOS



Native Android



Detecting Native vs Web: `hotwire_native_app?` ⚡

1. User-Agent Header

Hotwire Native mobile shells send custom HTTP User-Agent headers.

2. `hotwire_native_app?` Helper

Rails helper checks User-Agent and returns `true` for mobile apps.

3. HTML Root Data Attribute

Injects `data-hotwire-native` onto `<html>` root element.

`app/views/layouts/application.html.erb`

```
<!DOCTYPE html>
```

```
<html data-theme="silk" <%= "data-hotwire-native" if  
hotwire_native_app? %>>
```

```
<head>
```

```
  <%= stylesheet_link_tag "native" if hotwire_native_app? %>
```

```
</head>
```

```
<body class="bg-background text-content">
```

```
  <%= render "shared/navbar" %>
```

```
  <%= yield %>
```

```
</body>
```

```
</html>
```

Hiding Web Navbars: `hotwire-native:hidden` 🎨

Step 1

Step 2

Step 3

1. Root HTML Flag

Attach `data-hotwire-native` on `<html>`.

```
app/views/layouts/application.html.erb
<html <%= "data-hotwire-native" if hotwire_native_app? %>>
```

2. Custom Tailwind Variant

Define `@custom-variant hotwire-native`.

```
app/assets/tailwind/application.css
@custom-variant hotwire-native {
  html[data-hotwire-native] & { @slot; }
}
```

3. Apply Utility Class

Use `hotwire-native:hidden` in ERB view.

```
app/views/shared/_navbar.html.erb
<header class="sticky top-0 hotwire-native:hidden">
```

Dynamic Native Titles: `content_for(:title)`

Step 1

Step 2

Step 3

1. Set Title in View

Pass page title in ERB: `content_for :title`.

2. Render HTML `<title>`

Layout prints title dynamically in HTML head.

3. Auto Sync Native Header

Hotwire Native updates Swift & Kotlin top bars.

```
app/views/books/show.html.erb
```

```
<%= content_for :title, @book.title %>
```

```
app/views/layouts/application.html.erb
```

```
<title> <%= content_for?(:title) ? content_for(:title) :  
"LitLoop" %> </title>
```

```
Hotwire Native Bridge (iOS & Android)
```

Auto Sync

```
🍏 navigationItem.title = webView.title
```

```
📱 toolbar.title = webView.title
```

● LIVE DEMONSTRATION

DemoTime ⚡

Switching to iOS Simulator, Android Emulator & Rails Live
Server...

How the Interaction Works

1 User Taps a Link

In LitLoop, tapping a book card interrupts default web reload and hands navigation to Hotwire Native.

2 Native Nav First

The iOS or Android app pushes a new native screen with smooth native transitions instantly.

3 HTML Renders Inside

The native controller fetches HTML from `litloop.club/books/42` and renders it inside the native view.

LitLoop Feed

/books

📖 Currently Reading

👍 The Midnight Library

TAP!

`<a href="/books/42">The Midnight Lib...`

LINK CLICK INTERCEPTED

Intercepts link click to `/books/42` and passes URL to native navigator.

```
// Hotwire Native Delegate
navigator.route("https://litloop.club/b
```

What Else Can You Do With Hotwire Native?



Native Bridge Components

Strada

Control native Swift & Kotlin UI elements directly from HTML

`data-bridge` attributes.



Push Notifications & Deep Links

APNs / FCM

Handle APNs & FCM notifications natively and route deep links straight to Rails controllers.



Instant OTA Deployments

Zero Queue

Update copy, layouts, and feature bug fixes on your server live — with zero App Store review delays.



100% Native Fallback

SwiftUI / Compose

Need heavy camera filters or custom SDKs? Effortlessly drop down to full native Swift or Kotlin screens.

When **NOT** To Use Hotwire Native



3D Games & AR Apps

Pokémon GO, Pikmin Bloom

For GPU-intensive 3D graphics, real-time physics, or heavy AR camera engines.

Use Instead:

Unity / Metal Engine



Massive Mobile Teams

Dedicated Mobile Engineering Teams

When you have large budgets and dedicated, separate iOS & Android engineering orgs.

Use Instead:

Separate Native Stacks

💡 **Sweet Spot:** SaaS, Marketplaces, E-Commerce & Content Apps built by lean teams.

Maximum Leverage ↗



COSCUP 2026



Ruby Taiwan

Thank You! 🙏

Scan for presentation slide, and starter code.



netale.et/#talks